FOUNDING WAY RESEARCH LLC

Path to the Future

A 7-day measured study of AI-paired engineering

PRELIMINARY REPORT · MAY 2026 · FURTHER ANALYSIS FORTHCOMING

Abstract

The headline claim about AI-paired engineering — "20× faster" — is real and misleading at the same time, and this study set out to measure which parts are which. Over 7 calendar days, one engineer with 22 years of experience, working with Claude Code, designed and shipped a fully playable browser game from scratch. The measured compression against a traditional 3-person team came out to ~20.7× across the full project.

The point of the study is not the number. It is what the number rests on. The compression held only because an experienced engineer stayed in the loop at every step — specifying architecture, reviewing every diff, catching what a plausible-looking AI suggestion would have gotten wrong. It varied sharply by task: design and planning compressed 25–40×, while writing genuinely new engine code on a known surface dropped to 14–18×. And it covers only the work that was actually done — the study shipped zero hours of multi-playtester validation, accessibility audit, or cross-browser testing, all of which a real team would still owe.

The subject of this report is **experience plus AI as an offsetting capability** — what an experienced engineer can ship when AI handles execution and a human supplies judgment. The game is the research artifact: a controlled, commit-timestamped record, not the headline.

Scope of this study (read this first)

This is a study of **time-to-develop cost savings** — not complete product validation, development, and deployment. That distinction is load-bearing, and naming it up front is a strength of the study, not a caveat to apologize for.

What this study measures: the wall-clock time to design, build, and ship a working software artifact, with an experienced engineer steering an AI coding tool, recorded against a defensible traditional-team baseline.

What this study explicitly does **not** cover: market validation, stakeholder requirements gathering, user-story definition, multi-playtester validation, accessibility audits, cross-browser/device testing, second-engineer code review, or performance profiling. A fully-resourced team would burn 30–50% of its calendar on exactly this work. It is deferred here, not erased.

This scoping reinforces the central finding rather than weakening it. The compression is real *for the development work it covers* — and the work it does not cover is precisely the work that domain experience and engineering judgment, not raw AI throughput, are needed to do well. (See *The discipline problem moves to the self*, below.)

The research artifact: a game

Path to the Future is a top-down life simulator fluent in the grammar of NES and SNES games — Zelda's rooms, Final Fantasy's dialog boxes, Oregon Trail's event rolls, Monopoly's random-card

shrug — rendered in the flat-color stillness of Kentucky Route Zero, narrated in the dry register of Hitchhiker's Guide. Ten years. One career. Starting January 2020.

It's a fully playable browser game — not a demo — that hands you a decade and asks what you'd do with it. 2020 to 2030. Two-month “rooms” filled with real decisions, random events, NPCs, mini-games, and the quiet chaos of being a person in a complicated world.

Each room is a top-down space you physically move through using your keyboard. You bump into colleagues, pick up objects, dodge moving hazards, and eventually reach a door that makes you choose: stay late or go home? Take the promotion or protect your health? The world reacts. The stats shift. Life happens.

The game spans 60 playable levels across 10 years, with each year structured as one cinematic January narrative room and six playable months. Rooms grow progressively more challenging as the years advance — early rooms are open spaces; by 2027, you're threading through moving obstacles with real physics. (See *Appendix A: Feature Inventory* for full details.)

Resources

- GitHub: https://github.com/corby-github/path-to-the-future/
- Live game: https://corby-github.github.io/path-to-the-future/

The research behind it

At its core this was a research project: what can Claude Code (Opus 4, 1M context, Max account) actually build from scratch in a week — with an experienced software engineer (22 years) steering?

The study ran 7 calendar days: May 10 through May 16, 2026. Every session was tracked with a purpose-built Claude skill. Every line of code, every JSON event, every sprite, every piece of dialog — created from scratch during those sessions and pushed in real time to a GitHub repository to help keep the timestamps honest.

The project produced research artifacts (still being compiled), a living 2,400-line design document across 26 versions, and a real, shippable, playable game.

Velocity & compression

Across 18 distinct sittings over 7 calendar days, total time at the keyboard was 37h 11m — against an estimated 91 team-days of equivalent output from a modal 3-person team (1 senior engineer + 1 designer + 1 PM, roles overlapping heavily, occasionally expanding to 4–6 for content-heavy days).

Metric	Value
Wall time at keyboard	37h 11m
Equivalent team-days	~91 (range 64–99)
Compression ratio	~20.7× (range 14.5–22.5×)
Product work (of wall time)	~30.5h (~82%)
Review / playtest	~5.2h (~14%)

Metric	Value
Process tooling	~1.5h (~4%)
Compression on product time only	~25×

Of the 37.2h, roughly 82% was direct product work. The remainder was live human review of AI output (~14%) and process tooling — skill edits, research artifacts, time-log scheme refinements (~4%). GitHub commit history provides a clear, timestamped record of both timing and content — making the full 7-day split fully defensible.

Honest framing

The 20.7× compression number is real but load-bearing on five things staying true:

The domain was already shaped. The compression ratio held at 18–22× from Day 1 forward because the first sprint specced the architecture. Greenfield subsystem design compresses less aggressively — the project's own data shows this: Day 1 planning sat at 25–40× (design conversations compress hardest); Day 6 expert-tier engine work sat at 14–18× (real new code on a known surface).

Live human review ran concurrent, not deferred. Across all 18 sittings, the engineer was actively reviewing AI output between commits — playtest notes, copy write-throughs, real-time bug diagnosis. The compression only holds because review runs in the loop. The day a session ships AI output without live review is the day the number lies.

The traditional-team estimates exclude what a team would also do. Stakeholder review, multi-playtester validation, second-engineer code review, accessibility audits, cross-browser testing, performance profiling. Every sprint log enumerates these. They are deferred, not erased. A team would burn 30–50% of its calendar on them.

Process tooling has a real but small cost (~4%). Captured cleanly under boundary blocks where the scheme existed. Worth doing — the /research skill created on Day 4 morning paid back across 5 subsequent artifacts — but it doesn't ship the game and shouldn't inflate the product denominator.

Per-sprint compression is bounded by sprint shape. The narrowest measured ratio (Day 4 morning, 10–15×) was a sprint where process tooling was honestly subtracted. The widest was honestly downgraded after caveating in-sprint overhead. Both ends converge to the same band when the artifacts are honest. The most defensible single number is the project-level 20.7×, with 14–22× as the sensitivity band.

The catch worth naming. The project shipped 0 hours of multi-playtester validation, 0 hours of stakeholder coordination, 0 hours of accessibility audit, and 0 hours of cross-browser/device testing across the entire 37.2h. Those bills are real and deferred. The compression number is honest for the work that was done — not for the work a fully-resourced team would also do.

The discipline problem moves to the self

There is a failure mode this workflow introduces that a traditional team does not have, and it deserves naming alongside the velocity numbers: **scope creep, with no one to negotiate against but yourself.**

When execution is cheap, the constraint that used to live in the code moves into the operator. On a team, scope creep is a negotiation — a PM pushes back, an estimate gets challenged, a second engineer asks whether the feature is worth it. Working solo with an AI that will cheerfully build whatever you ask, that friction disappears. The discipline problem doesn't go away; it relocates from the codebase to the self. And the self is a bad referee.

This was felt directly. The study was roughly 34 hours of focused product time across four intense days, on top of a full-time engineering job — effectively a second full-time job for a week. The velocity is real: there were nights of six merged PRs in a single sitting. But velocity is exactly what makes scope discipline hard, because every "wouldn't it be cool if…" is suddenly an hour of work instead of a day, and the cost signal that normally kills marginal features is muted.

This is the sharpest illustration of the study's central distinction. Claude Code compresses coding, refactoring, and iteration enormously. It **cannot** compress quality judgment, product alignment, fit, or scope discipline — and it will actively erode the last one if the operator lets it. The 22 years of experience is what converts the AI from a liability into a tool: knowing which "cool" ideas are dead ends, which abstractions will compound, and when to stop. A notable companion observation: some of the roles people assumed AI would automate away first — product judgment, scope discipline, knowing what *not* to build — are turning out to be the ones it can't touch.

Scope & limitations

Path to the Future is a browser-based single-player game. It has no backend, no server, no database, no authentication, no API surface, no concurrency, no multi-threading, and no security model. The compression ratios reported here are scoped to exactly that — a frontend TypeScript/React application with local state persistence and a content-driven architecture. (See *Appendix B: Codebase and Technology Stack* for the full breakdown.)

This is not a claim about what AI-paired engineering looks like on distributed systems, microservices, high-concurrency backends, regulated APIs, or enterprise security surfaces. Those domains carry coordination, compliance, and architectural complexity that a single-player browser game simply doesn't have. The engineer running this study works on those systems professionally and is not confused about the difference.

What this study does claim: within its scope, the output is real, the timestamps are verifiable, and the compression numbers are defensible. A game is a legitimate software artifact — it has state management, a rendering loop, a content pipeline, an event system, persistence, and a shipping target. It is not a toy problem. It is also not a distributed system, and this report doesn't pretend otherwise.

Further analysis

This report covers the primary deliverable and velocity data. Additional research is in progress and will be compiled into a full report. Areas under active analysis include:

- **Friction points** — a detailed log of where the AI-paired workflow broke down, slowed, or required significant human intervention to recover.
- **Prompt engineering patterns** — what worked, what didn't, and how the prompting strategy evolved across 18 sittings.

- **Context window management** — strategies for working within and across context limits, including memory hardening, session handoffs, and plan usage across a Max account over 7 days.
- **Claude Code vs. Claude.ai** — how the two surfaces were used differently across the study.
- **Design doc as source of truth** — how a living 26-version design document functioned as the project's memory and spec simultaneously.
- **Skill development** — the purpose-built Claude skills created during the study, their evolution, and their measurable impact on velocity.
- **Content pipeline** — how 145 decisions and events were authored, reviewed, and integrated at pace — including the 150+ custom SVG icons designed, iterated, and wired into the game within the same window.
- **What breaks at scale** — honest assessment of where this workflow would degrade on a larger or more complex system.

Appendix A: Feature Inventory

From scratch, in 7 days: a complete game engine, two fully authored career packs, a procedural room generator, five mini-games, a stat and progression system, save/load persistence, a backward replay mechanic, cinematic transitions, a living title screen, an endgame recap, a credits system, a first-run tutorial, analytics instrumentation, 150+ custom SVG graphics, and 145 pieces of authored dialog and events — all committed to a GitHub repository and playable today on GitHub Pages.

Item	Count
Playable career packs	2
Starting classes	8
Tracked stats (HUD)	8
Playable levels	60
Room layout templates	35
Mini-games	5
Decisions	67
Events	78
Total decisions + events	145
Interactable NPCs and objects	28
Custom SVG graphics	150+

2 playable career packs — Software Engineer and Homeschool Parent — each with their own decisions, events, NPCs, room layouts, dialog, and era-specific flavor. Three more careers are scaffolded in the picker (Accounting, Nursing, Security/Police Officer) and ready for content. The

architecture externalizes career packs entirely: loosely coupled JSON manifests that plug into the engine, making new careers an authoring problem, not an engineering one.

8 selectable starting classes per career — from Novice to The Oracle — each with calibrated starting stats and XP. The Homeschool pack relabels all 8 tiers in its own register (Homeschooler Newbie through The Oracle) using the same engine, no code change.

8 tracked stats — The HUD tracks 8 live stats throughout every run: Burnout, Savings, Health, Network, Relationship, Technical Skill, Reputation, and XP — all visible at a glance via custom icons and updating in real time as you move through each room.

60 playable levels across 10 years (2020–2029), structured as 7 slots per year: one cinematic January and six playable months.

35 room layout templates across 5 complexity tiers — 13 simple, 4 easy, 8 medium, 6 hard, 4 expert — including static obstacle layouts, sine-wave moving hazard rooms, pong-style paddle gauntlets, and deterministic path sentinels. Rooms are procedurally generated from a seed combining your career, class, month, and macro game state — two players with different histories see different rooms.

5 mini-games — Blackjack, Code Review (spot the bug), Reaction Sprint (an alternating-direction stacker), Pong (vs. AI), and The Ultimate Question (the one with 42). Three appear at scheduled story beats; all five are accessible via an in-world arcade cabinet with a real-time XP throttle.

145 decisions and events — 67 decisions and 78 events across both career packs. A history-aware deduplication system ensures you're unlikely to see the same scenario twice in a short window.

28 interactable NPCs and objects — split across both career packs plus a universal arcade cabinet available to all — each with typewriter-reveal NPC dialog, pack-filtered placement, and palette-aware sprites.

150+ custom SVG graphics — 146 registered modal icons in the SWE pack alone (67 decision icons, 63 event icons, 5 minigame icons, plus supporting art), 27 unique interactable sprite tokens across both packs, and 8 custom stat icons. Every graphic is palette-aware and re-tints dynamically as the era shifts. The pandemic years drain color; the AI-boom years saturate.

A backward replay system — a second door in every room lets you walk back through your past decisions in read-only mode. Minigame results are recorded in history so replayed slots show a frozen summary. The era palette follows the viewed month, so walking back to 2020 brings the pandemic color grading with it.

A living title screen, career summary, and endgame recap — including a custom SVG trophy crown, a score derived from your final stats and decision history, a full career timeline showing every decision you made and what you chose, and a replay option guarded by a confirmation screen that reads: "If we do this, there's no going back. I know how fickle you can be."

Cinematic January transitions — narrative-only rooms that punctuate each year turn without interrupting the rhythm.

Analytics — GoatCounter wired with 11 pageview slugs and 4 custom events, gated behind DNT and environment flags. Dev builds never report.

Appendix B: Codebase and Technology Stack

Measured at close of study using cloc against all committed files:

Language	Files	Code	Comments	Blank
TypeScript	95	16,232	2,751	1,328
JSON	21	7,388	—	36
HTML	7	6,416	20	478
Markdown	4	2,091	—	452
JavaScript	3	188	36	22
CSS	1	92	81	19
Total	**131**	**32,407**	**2,888**	**2,335**

Summary

131 files. 32,407 lines of code. TypeScript makes up half the codebase; the JSON career pack data accounts for nearly a quarter — a direct reflection of how much of the game lives in content rather than engine.

Technology stack

TypeScript + React, browser-only, keyboard-first. Redux for persisted game state; React refs and hooks for 60fps player movement. SVG viewBox virtual coordinate system (1000×600 internal units) scaled to any screen. localStorage for saves with versioned state migration. GoatCounter for analytics.

Full report forthcoming. Raw logs, sprint artifacts, and commit history are available in the repository. All design documents, code, graphics, and dialog were created from scratch during the 7-day study window.